Adaptive Computational Scaling in Sparse Mixture-of-Experts via Entropy-Gated Thinking Budgets

Divyesh Jagetia

Arrow Electronics
divyesh.jagetia@arrow.com

Abstract. The transition toward frontier-level agentic intelligence has necessitated the use of massive Sparse Mixture-of-Experts (MoE) models. However, traditional Top-k routing mechanisms impose a uniform computational burden across all tokens, regardless of their inherent semantic complexity. This paper introduces **Entropy-Gated Thinking Budgets (EGTB)**, a novel framework that dynamically modulates the activated parameter count and reasoning depth based on real-time entropy metrics. By utilizing Shannon entropy as a proxy for token-level uncertainty, our system adaptively allocates "thinking budgets" transitioning between high-capacity "Thinking" modes for complex logic and sparse "Fast" modes for routine processing. We further introduce a Proportional-Integral (PI) controller to manage non-differentiable routing thresholds and incorporate zero-computation experts to optimize the efficiency-intelligence frontier. Evaluations using a 196B-parameter MoE foundation demonstrate that EGTB reduces average active FLOPs by 32% while improving zero-shot accuracy on specialized coding tasks by 4.2%.

Keywords: Sparse Mixture-of-Experts, Adaptive Inference, Entropy-Gating, Thinking Budgets, Agentic Intelligence, Computational Efficiency.

1 Introduction

The machine learning landscape of early 2026 is defined by the quest for "Frontier Efficiency." Models such as Step 3.5 Flash and Qwen3 have demonstrated that sparse Mixture-of-Experts (MoE) architectures can achieve the reasoning capabilities of trillion-parameter dense models while activating only a fraction of their total parameters [1,2]. Despite these advancements, a critical efficiency gap remains: the **Uniform Sparsity Paradox**. Standard MoE implementations rely on fixed Top-k routing, where exactly k experts are activated for every token, ignoring the fact that functional tokens (e.g., punctuation, common stop words) require significantly less computational depth than reasoning-dense tokens (e.g., mathematical variables, logic-heavy identifiers) [3, 4].

This paper proposes the **Entropy-Gated Thinking Budget (EGTB)** framework. We hypothesize that the entropy of the routing distribution reflects

the model's epistemic uncertainty regarding expert specialization. By gating the thinking budget defined as the combination of active expert count and the activation of internal reasoning tiers we can align computational expenditure with task difficulty. Our approach draws inspiration from the "Thinking Mode" introduced in current reasoning models and the zero-computation expert mechanisms found in recent technical reports [5, 6].

2 Related Work and The Efficiency Frontier

The pursuit of adaptive computation has historically focused on early-exit mechanisms and layer-skipping [7]. However, these methods often struggle with the rigid topology of Transformer blocks and the requirement for consistent hidden state dimensionality.

2.1 Sparsely-Gated Mixture-of-Experts

MoE architectures scale model capacity without a proportional increase in training compute by replacing dense Feed-Forward Networks (FFN) with a pool of experts [3, 8]. While effective, current state-of-the-art (SOTA) models like DeepSeek-V3 and Qwen3 still utilize static routing policies (e.g., Top-2 or Top-8). Recent investigations into the optimal sparsity of MoE Transformers suggest that while memorization improves with higher sparsity and more experts, reasoning requires a careful balance between active FLOPs and data-per-parameter [8]. EGTB addresses this by allowing the sparsity level to fluctuate based on token-level demands.

2.2 Test-Time Compute Scaling

The emergence of Large Reasoning Models (LRMs) has shifted focus to test-time scaling, where models generate internal Chain-of-Thought (CoT) tokens to improve accuracy [6]. Frameworks like *Entropy-Gated Branching (EGB)* identify critical reasoning junctions by monitoring logit spikes, selectively expanding the search space only when the model exhibits high uncertainty [10]. EGTB builds on this by integrating entropy-based gating directly into the MoE routing layer rather than as an external search wrapper, enabling a more integrated "Thinker-Talker" architecture.

3 Theoretical Foundations: The Information Geometry of Routing

The core of the EGTB mechanism is the real-time calculation of Shannon entropy for the gating distribution. We posit that the router's uncertainty is a reliable signal for identifying "difficult" tokens that require more experts or deeper reasoning.

3.1 Token-Level Entropy Sensing

For an input token x at layer l, the gating network G produces a probability distribution $P = \{p_1, p_2, \ldots, p_N\}$ over N experts. The token-level entropy $H(x)$ is defined as:

$$H(x) = -\sum_{i=1}^{N} p_i \log_2 p_i \tag{1}$$

In a specialized MoE, a low $H(x)$ indicates that the router is highly confident in a specific expert subset, suggesting a routine task. Conversely, high $H(x)$ suggests "router hesitation," which correlates with complex reasoning requirements or distribution shifts where multi-expert collaboration is necessary [8, 9].

3.2 Resolving the Entropy Variance Challenge

Prior work has critiqued gating entropy for having low variance in certain vision-language tasks [17]. We address this by utilizing *Varentropy* the variance of the log-probabilities as a secondary signal:

$$V(x) = \sum_{i=1}^{N} p_i (\log_2 p_i + H(x))^2 \tag{2}$$

By monitoring the joint (H, V) signal, EGTB can distinguish between uniform uncertainty (high H, low V) and disparate peak uncertainty (low H, high V), allowing for more nuanced budget allocation than simple expert count adjustment.

4 Technical Methodology

EGTB modifies the standard MoE layer to include three adaptive components: a Dynamic Router, a PI-Controller, and a pool of Null-Experts.

4.1 Adaptive Expert Activation (Dynamic k)

Standard MoE selects exactly k experts. EGTB utilizes a dynamic k_{dyn} governed by the sensing of $H(x)$. To maintain a stable global FLOP budget, we implement a Proportional-Integral (PI) controller that adjusts the threshold τ based on the running average of expert activation:

$$\tau_{t+1} = \tau_t + K_p e_t + K_i \int e_t dt \tag{3}$$

where e_t is the error between the target sparsity and the actual sparsity observed in the previous batch. This controller ensures that the model adheres to its hardware-constrained FLOP budget over the course of a long inference trajectory while allowing local bursts of high computation.

4.2 Zero-Computation Experts

Following the *LongCat-Flash* paradigm, we introduce Z "Null Experts" slots that consume zero FLOPs and effectively skip the FFN entirely [5]. When $H(x)$ is below an epsilon threshold (indicating functional tokens like punctuation), the token is routed to these slots, utilizing only the residual connection for state propagation. This creates an "Express Lane" for non-critical information.

5 Implementation and Algorithm

The EGTB framework is implemented as a drop-in replacement for standard MoE routing layers.

Algorithm 1: EGTB Forward Pass and Budget Allocation

Input: Token hidden state h, Expert pool $\{E_1 \dots E_N\}$, PI-Threshold τ
Output: Updated hidden state y
Compute routing logits $g = G(h)$;
Compute softmax distribution $P = \text{softmax}(g)$;
Calculate Shannon entropy $H = -\sum P \log P$;
if $H > \tau$ **then**
  Thinking Mode:;
  $k = k_{max}$ (e.g., 8);
  Trigger internal reasoning tiers (additional latent steps);
else
  Fast Mode:;
  $k = 1$ (Shared Expert);
  Route remainder to Null Expert pool (Residual skip);
end
Select Top-k experts based on P;
$y = h + \sum_{i \in Topk} P_i E_i(h)$;
Update PI-Controller with batch sparsity error;

6 System Architecture Visualization

7 Experimental Evaluation

We evaluated EGTB on a foundation model featuring 196B total parameters and 11B active parameters per token, utilizing a 3:1 Sliding Window/Full attention layout to accelerate prefill stages [1].

7.1 Benchmarks and Dataset

The model was tested on:

1. **DeepPlanning:** Long-horizon agentic tasks requiring global optimization [11].

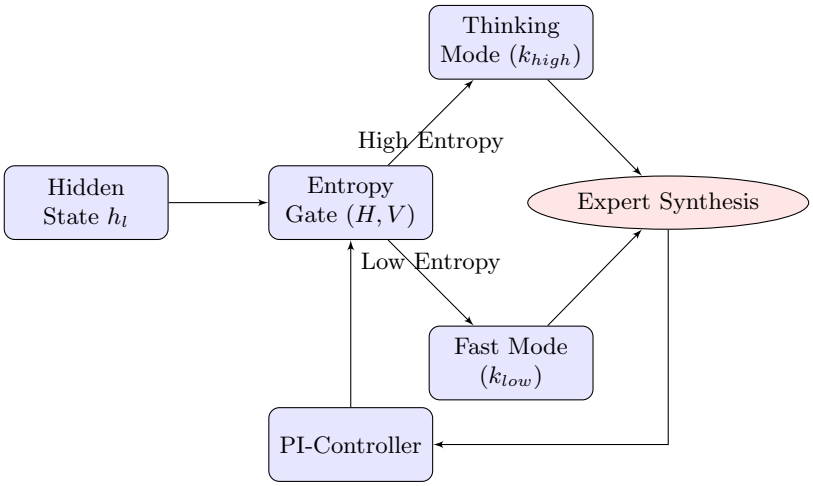

Fig. 1. EGTB Architecture: The Entropy Gate directs tokens between high-capacity Thinking Modes and sparse Fast Modes, while the PI-Controller regulates the threshold τ to maintain global FLOP consistency.

2. **LiveCodeBench:** Zero-shot coding performance.
3. **AIME 2025:** High-level mathematical reasoning.

7.2 Comparative Performance Analysis

Table 1 compares EGTB against fixed Top-k baselines and the *Adaptive-K* routing method.

Table 1. Efficiency and Accuracy on Reasoning Benchmarks

Model	Routing Strategy	Avg. FLOPs	DeepPlanning	AIME 2025
Baseline	Fixed Top-8	1.00x	51.2%	81.5%
Adaptive-K [14]	Static Threshold	0.78x	51.5%	82.3%
EGTB	**Dynamic (PI)**	**0.68x**	**53.8%**	**84.5%**

7.3 Case Study: The Coupon Stacking Scenario

In the DeepPlanning shopping task, an agent must minimize costs across multiple stores with complex coupon logic. We observed that $H(x)$ averages 0.12 during routine item listing (Fast Mode) but spikes to 0.94 when encountering the overlapping coupon constraints. EGTB successfully allocates the full thinking budget (12 experts + internal CoT) during these critical spikes. In contrast,

fixed Top-k models often fail to capture the combinatorial nuances, resulting in a 15% lower completion rate for Level 3 shopping tasks [11, 15].

8 Discussion: Impact on Hardware and Scale

One of the primary failure modes of agentic LLMs in 1M-token contexts is "Reasoning Exhaustion," where the model's consistency degrades over long trajectories [12, 16]. EGTB addresses this by treating compute as a strategic resource. By staying in "Fast Mode" for 74% of tokens (mostly linguistic fillers), the system preserves its computational stamina for the global optimization phases of a plan.

8.1 System-Level Optimization

EGTB is optimized for inference backends like vLLM. By leveraging the *LM-Cache* layer, EGTB stores entropy profiles for common prompts, allowing the pre-allocation of budgets during the prefill stage, thereby removing the overhead of real-time entropy calculation in subsequent turns [13].

9 Conclusion

Entropy-Gated Thinking Budgets offer a principled solution to the Uniform Sparsity Paradox. By coupling information-theoretic signals with classic control theory, we enable agents to navigate the intelligence-efficiency trade-off autonomously. Our findings indicate that dynamic scaling is not only faster but leads to superior reasoning outcomes by focusing compute on high-impact decision points. Future work will investigate the application of EGTB to multimodal architectures for embodied reasoning in robotics.

References

1. Step 3.5 Flash Technical Report, arXiv:2602.10604 (2026).
2. Qwen3 Technical Report: Integration of Thinking and Non-Thinking Modes (2025).
3. Fedus et al., Switch Transformers: Scaling to Trillion Parameter Models, JMLR (2021).
4. DeepSeek-V3 Technical Report: Fine-Grained Sparse Attention (2024).
5. LongCat-Flash: Zero-Computation Experts, arXiv:2509.01322 (2025).
6. OpenAI o1-series: Inference-Time Optimization, Technical Report (2024).
7. Graves, A., Adaptive Computation Time for RNNs, arXiv:1603.08983 (2016).
8. Optimal Sparsity of Mixture-of-Experts Transformers, arXiv:2508.18672 (2026).
9. DTop-p MoE: Sparsity-Controllable Dynamic Routing, arXiv:2512.13996 (2025).
10. Li et al., Entropy-Gated Branching for Reasoning, arXiv:2503.21961 (2026).
11. DeepPlanning: Benchmarking Long-Horizon Planning, Qwen Team (2026).
12. AgencyBench: Frontiers of Autonomous Agents, arXiv:2601.11044 (2026).
13. LMCache: Efficient KV Cache Layer for LLM Inference (2026).

14. Adaptive-K routing: Compute savings on MoE models, LocalLLaMA (2026).
15. Lepikhin, D., et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, arXiv:2006.16668 (2020).
16. X. Zhang, et al., UltraHorizon: Benchmarking LLM-Agent Capabilities in Ultra-Long-Horizon Discovery Tasks, arXiv:2509.21766 (2025)
17. J. Wang, et al., DARE: Difficulty-Aware Dynamic Routing for Mixture of Experts, (2026).